BASES DE DATOS 2

Seguridad y control de acceso



GRANT · REVOKE · vistas · roles · seguridad a nivel de fila (RLS)

Ingeniería de Sistemas · Clase 07

FACULTAD DE CIENCIAS EXACTAS
UNIVERSIDAD NACIONAL DEL CENTRO DE LA PROVINCIA DE BUENOS AIRES

Contenido de la clase

01

Seguridad de la información

Qué protegemos, de qué, y qué parte resuelve el SGBD

02

Control de acceso discrecional

GRANT y REVOKE: privilegios, propietarios, grafo de concesión

03

Vistas como mecanismo de seguridad

Exponer solo algunas columnas y filas

04

Roles

Agrupar privilegios; diseñar una jerarquía de roles

05

PostgreSQL en la práctica

Atributos de rol, esquemas, privilegios por defecto, inspección

06

Seguridad a nivel de fila

Row Level Security: políticas que filtran filas (el "VPD" de PostgreSQL)

Seguridad de la información

Proteger los datos contra accesos no autorizados involucra mucho más que el SGBD. Contempla, al menos, estos aspectos:

Éticos y legales

Quién tiene derecho a ver qué. Normativa de protección de datos personales (en Argentina, Ley 25.326).

Políticas organizacionales

Definición explícita de quién accede a qué información y con qué fin.

Operacionales

Generación, resguardo y rotación de claves; gestión de cuentas.

Cifrado

Datos confidenciales cifrados en reposo (disco, backups) y en tránsito (TLS).

Ataques directos e indirectos

Inferir información confidencial a partir de datos públicos; inyección de SQL a través de la aplicación.

En esta clase

El subsistema de autorización del SGBD: GRANT / REVOKE, vistas, roles y políticas por fila.

Seguridad en la base de datos

El SGBD provee un subsistema de seguridad y autorización. El DBA debe asegurar una política de acceso clara y consistente:

Decidir
quién accede a la BD y qué puede hacer sobre cada objeto

Garantizar
que no haya accesos no autorizados

No impedir
el acceso a los datos a los usuarios habilitados

¿Quiénes?
Qué usuarios o roles. Personas, aplicaciones, procesos.

¿Qué datos?
Tablas, vistas, columnas... ¿y filas?

¿Qué operaciones?
Solo consulta, modificación de datos, cambios de estructura.

Autenticación
¿Sos quien decís ser? Cuenta y clave, certificados, LDAP/Kerberos. En PostgreSQL se configura en pg_hba.conf.

Autorización
¿Qué podés hacer una vez adentro? GRANT / REVOKE, roles, vistas, políticas por fila. Es el tema de esta clase.

El administrador de la base de datos (DBA)

Responsable de la seguridad global de la BD

Posee una cuenta privilegiada en el SGBD (superusuario).

Organiza los usuarios según la política de la organización

Crea las cuentas de usuario y los grupos (roles).

Otorga privilegios

GRANT: decide quién puede hacer qué, sobre qué objetos.

Revoca privilegios concedidos

REVOKE: cuando cambia la política o el usuario cambia de función.

Controla la actividad

El SGBD verifica cuenta y clave al conectar y registra las acciones de cada sesión (logs, auditoría).

En PostgreSQL

`postgres`

es el superusuario inicial: no tiene restricciones, ni siquiera las políticas RLS.

`CREATE ROLE`

crea usuarios y grupos; todo es un rol.

`pg_hba.conf`

define quién puede conectarse, desde dónde y con qué método de autenticación.

Regla de oro

ni las personas ni la aplicación operan con el superusuario: cada uno con su rol y sus privilegios mínimos.

Métodos de control de acceso

El acceso a la BD se basa en otorgar y revocar privilegios sobre los objetos de la BD mediante:

Control de acceso discrecional (DAC)

El propietario de cada objeto decide quién accede y con qué operaciones, y puede delegar esa capacidad (WITH GRANT OPTION). Granularidad: objeto y columna. Es el modelo del estándar SQL y de PostgreSQL.

```
GRANT SELECT ON cliente TO ana;
```

Control de acceso basado en roles (RBAC)

Los privilegios se agrupan en roles y los usuarios reciben roles. Cambiar el rol cambia los privilegios de todos sus miembros. Escala cuando hay muchos usuarios.

```
GRANT ventas_lectura TO ana;
```

Y dentro de una tabla, ¿qué filas?

Ninguno de los dos lo resuelve: para eso están las políticas de seguridad a nivel de fila (RLS), última sección.

GRANT: otorgar privilegios

```
GRANT privilegio [, ...] ON objeto [, ...]
   TO rol [, ...] [ WITH GRANT OPTION ];
```

privilegio — derecho a ejecutar una operación sobre el objeto: SELECT, INSERT, UPDATE, ...

objeto — tabla, vista, columna, secuencia, esquema, función, base de datos.

rol — usuario o rol que la BD reconoce, o PUBLIC: todos los usuarios, incluso los futuros.

WITH GRANT OPTION — quien recibe el privilegio puede a su vez concederlo a otros.

```
-- ejemplos
GRANT SELECT ON cliente TO ana;
GRANT SELECT, INSERT ON cliente, factura
   TO bruno;
GRANT UPDATE (telefono) ON cliente TO ana;
GRANT SELECT ON cliente TO PUBLIC;
GRANT SELECT ON cliente TO carla
   WITH GRANT OPTION;
```



Grafo de concesión

Un mismo privilegio puede propagarse a varios usuarios y recibirse de varios otorgantes. El SGBD registra quién concedió cada uno: eso importa al revocar.

Privilegios sobre objetos en PostgreSQL

Privilegio	Se aplica a	Permite
`SELECT`	tabla, vista, columna, secuencia	leer filas (incluye columnas agregadas después con ALTER TABLE)
`INSERT`	tabla, columna	agregar filas (con columnas: solo asignar valor a esas columnas)
`UPDATE`	tabla, columna, secuencia	modificar filas; un WHERE que lee columnas requiere además SELECT
`DELETE`	tabla	eliminar filas; ídem SELECT si el WHERE lee columnas
`TRUNCATE`	tabla	vaciar la tabla completa
`REFERENCES`	tabla, columna	definir claves foráneas que apunten a la tabla
`TRIGGER`	tabla	crear triggers sobre la tabla
`USAGE`	esquema, secuencia, tipo	"entrar" al esquema y ver sus objetos; usar nextval / currval
`EXECUTE`	función, procedimiento	invocarla
`CREATE`	base de datos, esquema	crear esquemas en la BD / crear objetos en el esquema
`CONNECT`	base de datos	conectarse a esa base de datos



Privilegios de columna: SELECT, INSERT, UPDATE y REFERENCES admiten (columna, ...) para limitar el privilegio a columnas específicas.

Dos niveles de asignación de privilegios

En los libros (Elmasri-Navathe)

Nivel de cuenta

Capacidades de cada usuario, independientes de las relaciones: CREATE SCHEMA, CREATE TABLE, CREATE VIEW, ALTER, DROP, ...

Nivel de relación

Privilegios sobre relaciones particulares: SELECT, INSERT, UPDATE, DELETE, REFERENCES (con o sin lista de columnas).

En PostgreSQL

Atributos del rol

LOGIN, PASSWORD, SUPERUSER, CREATEDB, CREATEROLE, BYPASSRLS, INHERIT. Se definen con CREATE ROLE / ALTER ROLE, no con GRANT.

Privilegios sobre objetos

Se otorgan con GRANT. "Poder crear tablas" no es un atributo: es USAGE + CREATE sobre un esquema concreto.

```
CREATE ROLE ana LOGIN PASSWORD 'secreto';      -- atributos: quién puede conectarse
ALTER ROLE ana CREATEDB;                        -- atributo: puede crear bases de datos
GRANT USAGE, CREATE ON SCHEMA ventas TO ana;    -- privilegios: puede crear objetos en ventas
```



No existe GRANT CREATE TABLE TO ana en PostgreSQL: esa sintaxis es de otros motores (Oracle, SQL Server). Crear tablas se controla por esquema.

REVOKE: retirar privilegios

```
REVOKE [ GRANT OPTION FOR ] privilegio [, ...]
   ON objeto [, ...]
   FROM rol [, ...] [ CASCADE | RESTRICT ];
```

```
REVOKE SELECT ON cliente FROM ana;
REVOKE GRANT OPTION FOR SELECT ON cliente
   FROM carla;
REVOKE UPDATE ON cliente FROM bruno CASCADE;
```

Propietarios

El propietario de un objeto tiene todos los privilegios sobre él (y puede concederlos); nadie se los puede revocar. El propietario del esquema puede crear, alterar y eliminar objetos en él. Se transfiere con ALTER TABLE t OWNER TO rol.

GRANT OPTION FOR

Quita solo la capacidad de propagar el privilegio. El usuario conserva el privilegio sobre el objeto.

CASCADE

Revoca el privilegio al usuario y a todos los que lo recibieron a través de él. Si alguien lo recibió también por otro camino, lo conserva.

RESTRICT (por defecto en PostgreSQL)

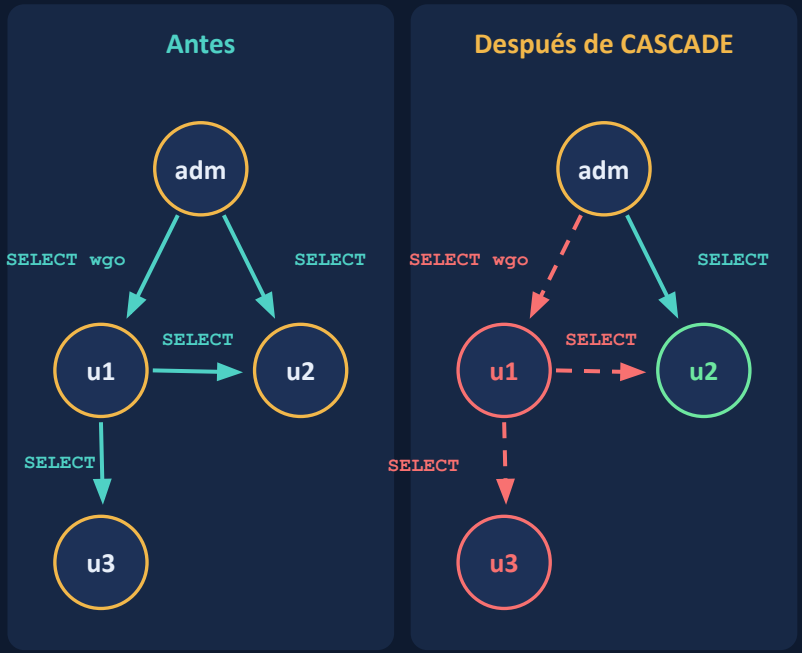
Si quedaran privilegios "colgados" (dependientes), el REVOKE falla: ERROR: dependent privileges exist.

El grafo de concesión: RESTRICT vs. CASCADE

```
-- adm es el propietario de T1
adm: GRANT SELECT ON T1 TO u1 WITH GRANT OPTION;
adm: GRANT SELECT ON T1 TO u2;
u1:  GRANT SELECT ON T1 TO u3;
u1:  GRANT SELECT ON T1 TO u2;

adm: REVOKE SELECT ON T1 FROM u1;    -- RESTRICT
ERROR:  dependent privileges exist
HINT:  Use CASCADE to revoke them too.

adm: REVOKE SELECT ON T1 FROM u1 CASCADE;
```



Resultado: u1 y u3 pierden SELECT sobre T1. u2 lo conserva: también lo había recibido directamente de adm.

Para resolver: ¿qué privilegios conserva cada uno?

```
-- A es propietario del esquema y crea T1 y T2
A: GRANT SELECT ON T1, T2 TO B;
A: GRANT SELECT ON T1, T2 TO C WITH GRANT OPTION;
C: GRANT SELECT ON T1 TO B, D;

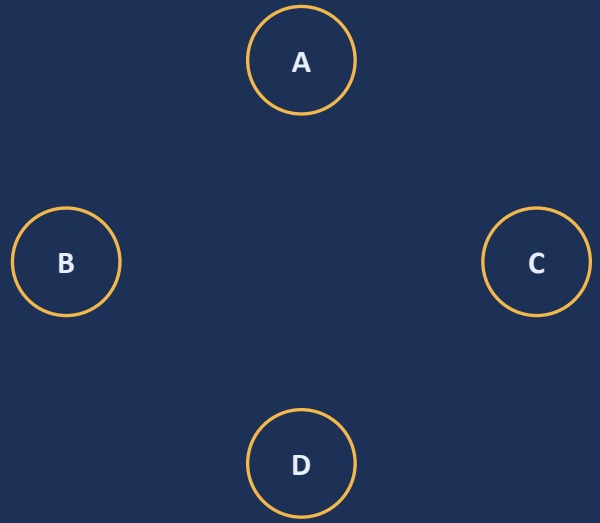
A: REVOKE SELECT ON T1 FROM C CASCADE;
```

Para pensar

1. ¿Qué puede hacer cada usuario sobre T1 y sobre T2 después del REVOKE?
2. ¿Qué habría pasado si A no escribía CASCADE?
3. ¿Y si en lugar de eso A ejecuta REVOKE GRANT OPTION FOR SELECT ON T1 FROM C CASCADE?

Dibujá el grafo de concesión

una arista por cada GRANT: quién, a quién, qué privilegio, con o sin grant option



Vistas como mecanismo de seguridad

Una vista expone solo una parte de los datos. Se conceden privilegios sobre la vista, no sobre la tabla: el usuario nunca toca la tabla base.

```
CREATE VIEW empleado_publico AS
  SELECT legajo, nombre, sector
  FROM empleado;      -- sin sueldo ni CUIL

GRANT SELECT ON empleado_publico TO consulta;
-- consulta NO tiene SELECT sobre empleado

CREATE VIEW mis_facturas AS
  SELECT * FROM factura
  WHERE vendedor = current_user;   -- filas
```

Ocultar columnas y filas

La proyección esconde columnas sensibles; la selección esconde filas. Es la forma clásica de "seguridad por filas"... con límites (última sección).

Requisitos para crearla

Hay que tener SELECT sobre todas las relaciones base (tablas o vistas) que la definen.

Quién necesita qué privilegio

En PostgreSQL, el acceso a las tablas base se verifica con los privilegios del dueño de la vista, no de quien consulta. Por eso "consulta" puede leer la vista sin ningún privilegio sobre empleado.

Vistas y privilegios: estándar SQL vs. PostgreSQL

Situación	Estándar SQL (Date, Elmasri)	PostgreSQL
Conceder privilegios sobre la vista	El creador solo puede otorgar SELECT WITH GRANT OPTION sobre la vista si lo tiene sobre cada relación base.	El dueño de la vista concede lo que quiera sobre ella; las tablas base se verifican con sus propios privilegios al ejecutar la consulta.
El creador pierde SELECT sobre una relación base	La vista se elimina (y con ella los privilegios que otros habían recibido sobre ella).	La vista sigue existiendo. Cualquier consulta sobre ella falla: permission denied for table empleado.
Vista actualizable	El creador tiene INSERT / UPDATE / DELETE sobre la vista si los tiene sobre las relaciones base.	El dueño necesita esos privilegios en las tablas base; a los usuarios solo se les conceden sobre la vista.
Verificar con el usuario que consulta	No contemplado.	CREATE VIEW ... WITH (security_invoker = true) desde PostgreSQL 15: los permisos se comprueban con quien consulta.



`security_barrier`

Una vista con WHERE no garantiza por sí sola que las filas ocultas no se "filtren": una función maliciosa en la consulta del usuario podría evaluarse antes que el filtro. CREATE VIEW ... WITH (security_barrier = true) fuerza a aplicar el filtro primero.

Control de acceso basado en roles

Con muchos usuarios, administrar GRANT y REVOKE uno por uno se vuelve inmanejable. SQL:1999 incorpora roles; PostgreSQL adhiere.

```
CREATE ROLE nombre_rol [ atributos ];
GRANT rol [, ...] TO rol [, ...] [ WITH ADMIN OPTION ];
```

```
CREATE ROLE ventas_lectura;    -- grupo, sin LOGIN
GRANT USAGE ON SCHEMA ventas TO ventas_lectura;
GRANT SELECT ON ALL TABLES IN SCHEMA ventas
   TO ventas_lectura;

GRANT ventas_lectura TO ana, bruno;    -- membresía
GRANT ventas_lectura TO carla WITH ADMIN OPTION;
```



Rol = conjunto de privilegios

Un usuario puede tener varios roles. Si cambian los privilegios del rol, cambian para todos sus miembros.



En PostgreSQL todo es un rol

Un "usuario" es un rol con LOGIN (CREATE USER ≡ CREATE ROLE ... LOGIN). Los roles de grupo se crean sin LOGIN. Se puede conceder un rol a usuarios, a otros roles o a PUBLIC.



WITH ADMIN OPTION

El miembro puede conceder y revocar ese rol a otros. No existe un rol especial "ADMIN": crear y eliminar roles es el atributo CREATEROLE.

Revocar roles

```
REVOKE [ ADMIN OPTION FOR ] rol [, ...]
   FROM rol [, ...] [ CASCADE | RESTRICT ];
```

```
REVOKE ventas_lectura FROM ana;
-- ana deja de ser miembro: pierde lo que heredaba

REVOKE ADMIN OPTION FOR ventas_lectura FROM carla;
-- carla sigue siendo miembro, pero ya no puede
-- conceder el rol a otros

REVOKE SELECT ON ALL TABLES IN SCHEMA ventas
   FROM ventas_lectura;
-- el rol sigue existiendo, sin ese privilegio;
-- afecta a todos sus miembros a la vez
```

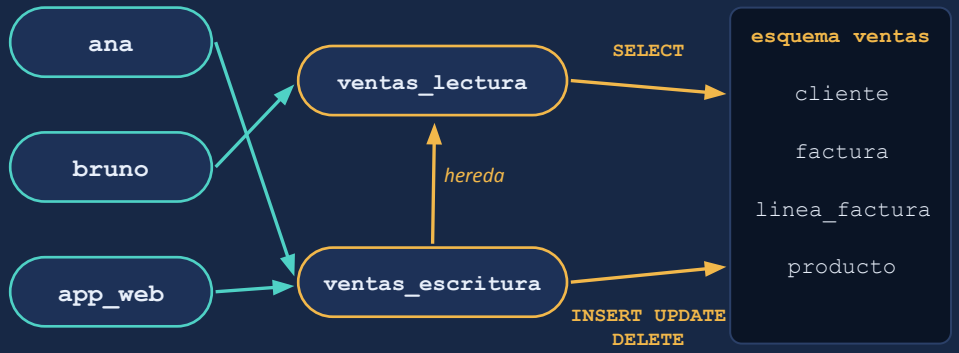
Membresía ≠ privilegio

Revocar el rol a un usuario lo saca del grupo. Revocar un privilegio al rol se lo quita al grupo entero. Son dos operaciones distintas con el mismo verbo.

Detalles

Los privilegios del propietario de un objeto no se pueden revocar. CASCADE / RESTRICT funcionan como en REVOKE de privilegios. Desde PostgreSQL 16 la membresía tiene tres opciones independientes: ADMIN, INHERIT y SET.

Diseñar con roles: personas, grupos, objetos



Principio de mínimo privilegio: cada rol recibe solo lo que necesita. Nadie opera con el superusuario; la aplicación se conecta con su propio rol; las personas reciben roles de grupo, no privilegios sueltos.

```
-- 1. roles de grupo
CREATE ROLE ventas_lectura;
CREATE ROLE ventas_escritura;
GRANT ventas_lectura TO ventas_escritura;

-- 2. privilegios sobre el esquema
GRANT USAGE ON SCHEMA ventas
   TO ventas_lectura;
GRANT SELECT ON ALL TABLES
   IN SCHEMA ventas TO ventas_lectura;
GRANT INSERT, UPDATE, DELETE
   ON ALL TABLES IN SCHEMA ventas
   TO ventas_escritura;

-- 3. personas y aplicaciones
CREATE ROLE ana LOGIN PASSWORD '...';
GRANT ventas_escritura TO ana;
CREATE ROLE app_web LOGIN PASSWORD '...';
GRANT ventas_escritura TO app_web;
```

PostgreSQL: detalles que importan

Trampas frecuentes

ON ALL TABLES alcanza solo a las tablas que ya existen

Para las futuras: ALTER DEFAULT PRIVILEGES IN SCHEMA ventas GRANT SELECT ON TABLES TO ventas_lectura;

PostgreSQL 15: el esquema public ya no es de todos

PUBLIC perdió el privilegio CREATE sobre public. Un usuario nuevo no puede crear tablas ahí salvo GRANT explícito.

Semántica que sorprende

REVOKE es RESTRICT por defecto. UPDATE y DELETE con WHERE necesitan también SELECT. Sin USAGE sobre el esquema, no se llega a la tabla.

Cómo inspeccionar

```
\du              -- roles y atributos
\dn+             -- esquemas y privilegios
\dp cliente      -- privilegios de la tabla
   ana=r/adm       adm dio a ana: r
   bruno=arwd/adm  a= INSERT r=SELECT
                   w= UPDATE d=DELETE
   carla=r*/adm    * = con grant option
\d+ cliente      -- incluye políticas RLS

SELECT has_table_privilege(
   'ana', 'cliente', 'SELECT');
SELECT * FROM
   information_schema.role_table_grants
 WHERE grantee = 'ana';
```

Seguridad a nivel de fila: ¿qué filas ve cada uno?

GRANT y las vistas deciden sobre objetos y columnas. ¿Y si cada vendedor solo debe ver sus propios clientes, con la misma tabla y las mismas consultas?

Filtrar en la aplicación

WHERE vendedor = :usuario en cada consulta del código.

Cualquier otra vía de acceso lo evita: psql, un reporte, una herramienta de BI, un bug.

Una vista por usuario

CREATE VIEW mis_clientes ... WHERE vendedor = current_user.

No escala: hay que mantenerla por tabla, obligar a usarla y proteger la tabla base.

Row Level Security

Una política sobre la tabla; el motor la aplica a toda consulta, venga de donde venga.

Funciona con SELECT, INSERT, UPDATE y DELETE. Nativa en PostgreSQL desde 9.5.

El mismo concepto en otros motores

Oracle	Virtual Private Database (VPD), paquete DBMS_RLS	**SQL Server**	Row-Level Security (security policies + predicate functions)	**PostgreSQL**	Row Level Security (CREATE POLICY), desde 9.5

Cómo funciona: privilegios primero, políticas después




```
ALTER TABLE cliente ENABLE ROW LEVEL SECURITY;
-- activa RLS; sin políticas, nadie ve filas

CREATE POLICY nombre ON tabla
   [ AS { PERMISSIVE | RESTRICTIVE } ]
   [ FOR { ALL | SELECT | INSERT | UPDATE | DELETE } ]
   [ TO rol [, ...] ]
   [ USING ( condicion ) ]       -- filas existentes
   [ WITH CHECK ( condicion ) ]; -- filas nuevas
```

USING

Filtra las filas existentes: cuáles se ven (SELECT), se modifican (UPDATE) o se borran (DELETE).

WITH CHECK

Valida las filas nuevas o modificadas: qué se puede insertar o en qué puede convertirse una fila al actualizarla.

Denegar por defecto

Con RLS activo y sin políticas aplicables, la tabla parece vacía y no acepta cambios. RLS se suma a GRANT, no lo reemplaza.

Ejemplo: cada vendedor ve solo sus clientes

```
CREATE TABLE cliente (
  id serial PRIMARY KEY, nombre text, vendedor name);
GRANT SELECT, INSERT, UPDATE ON cliente TO ventas;
ALTER TABLE cliente ENABLE ROW LEVEL SECURITY;

CREATE POLICY propio ON cliente TO ventas
   USING (vendedor = current_user)
   WITH CHECK (vendedor = current_user);

SET ROLE ana;                  -- ana es miembro de ventas
SELECT * FROM cliente;         -- 2 filas: solo las de ana
UPDATE cliente SET nombre = 'Acme SA' WHERE id = 2;
-- UPDATE 0: la fila 2 no existe para ana
INSERT INTO cliente (nombre, vendedor) VALUES ('Nuevo', 'bruno');
ERROR:  new row violates row-level security policy
        for table "cliente"

SET ROLE bruno;
SELECT count(*) FROM cliente;  -- 1: solo Acme
RESET ROLE;                    -- vuelve al dueño
```

Tabla cliente (lo que ve el dueño)

id	nombre	vendedor
1	**Lácteos Tandil**	**ana**
2	Acme	bruno
3	**Molinos Sur**	**ana**
4	Textil Norte	carla

filas que ve (y puede tocar) ana

Transparente para la app

Las consultas no cambian: SELECT * FROM cliente devuelve distinto según quién lo ejecute. Ni siquiera hay que saber que la política existe.

Row Level Security: detalles que hay que saber

Quién no es filtrado

El dueño de la tabla (salvo ALTER TABLE ... FORCE ROW LEVEL SECURITY), los superusuarios y los roles con el atributo BYPASSRLS.

Qué condición aplica cada comando

SELECT → USING · INSERT → WITH CHECK · UPDATE → USING (filas a tocar) y WITH CHECK (resultado) · DELETE → USING.

Varias políticas

Las PERMISSIVE se combinan con OR; las RESTRICTIVE con AND sobre el resultado. Si ninguna aplica al rol o al comando: ninguna fila.

Usuarios de la aplicación

Si la app se conecta con un único rol, pasa la identidad por sesión: SET app.usuario = 'ana' y en la política current_setting('app.usuario').

Rendimiento y fugas

La condición se agrega a cada consulta: indexar la columna filtrada. El planificador evalúa las políticas antes que las funciones no LEAKPROOF del usuario.

Cómo inspeccionar

\d+ tabla lista las políticas. Catálogo: SELECT * FROM pg_policies; Deshabilitar: ALTER TABLE ... DISABLE ROW LEVEL SECURITY.

Virtual Private Database (Oracle) vs. RLS (PostgreSQL)

	Oracle VPD	PostgreSQL Row Level Security
Cómo se define	Una función PL/SQL devuelve un predicado (texto); se registra con DBMS_RLS.ADD_POLICY sobre la tabla.	CREATE POLICY con una expresión SQL declarativa: USING / WITH CHECK.
Identidad del usuario	Contexto de aplicación: SYS_CONTEXT('USERENV', 'SESSION_USER') o contextos propios.	current_user, session_user, o parámetros de sesión: current_setting('app.usuario').
Alcance	Por sentencia (SELECT, INSERT, UPDATE, DELETE); además puede enmascarar columnas (column masking).	Por comando y por rol. No enmascara columnas: para eso, privilegios de columna o vistas.
Quién la evita	SYS y usuarios con EXEMPT ACCESS POLICY.	Dueño de la tabla (sin FORCE), superusuarios, roles con BYPASSRLS.
Desde	Oracle 8i (1999).	PostgreSQL 9.5 (2016).



Idea común: el motor agrega automáticamente el predicado a cada consulta sobre la tabla. El usuario no puede evitarlo desde SQL, cualquiera sea la herramienta con la que se conecte.

En síntesis



Autenticación ≠ autorización

pg_hba.conf y LOGIN deciden quién entra; GRANT, roles y RLS deciden qué puede hacer.



Mínimo privilegio

Nadie opera con el superusuario. Cada persona y cada aplicación con su rol.



Roles de grupo

Los privilegios van a roles sin LOGIN; las personas reciben membresías. ALTER DEFAULT PRIVILEGES para lo futuro.



Vistas para columnas

Exponer solo lo necesario; en PostgreSQL los permisos base se verifican con el dueño de la vista.



RLS para filas

Políticas sobre la tabla, aplicadas a toda consulta: el "VPD" de PostgreSQL.



Revisar y auditar

\du, \dp, \d+, pg_policies. REVOKE es RESTRICT por defecto: el grafo de concesión importa.

Bibliografía

Libros

Date, C. J.

An Introduction to Database Systems, 8.ª ed., Addison-Wesley, 2004, cap. 17. (7.ª ed. en español, 2001, cap. 16.)

Elmasri, R.; Navathe, S.

Fundamentals of Database Systems, 7.ª ed., Pearson, 2016, cap. 30. (6.ª ed., Addison-Wesley, 2011, cap. 24.)

Ramakrishnan, R.; Gehrke, J.

Database Management Systems, 3.ª ed., McGraw-Hill, 2003, cap. 21.

Documentación

PostgreSQL, manual oficial

Privileges · Row Security Policies · Database Roles · GRANT · REVOKE · CREATE POLICY · CREATE ROLE · ALTER DEFAULT PRIVILEGES · psql (\dp, \du).

Oracle Database Security Guide

"Using Oracle Virtual Private Database to Control Data Access".

Microsoft SQL Server docs

"Row-Level Security".